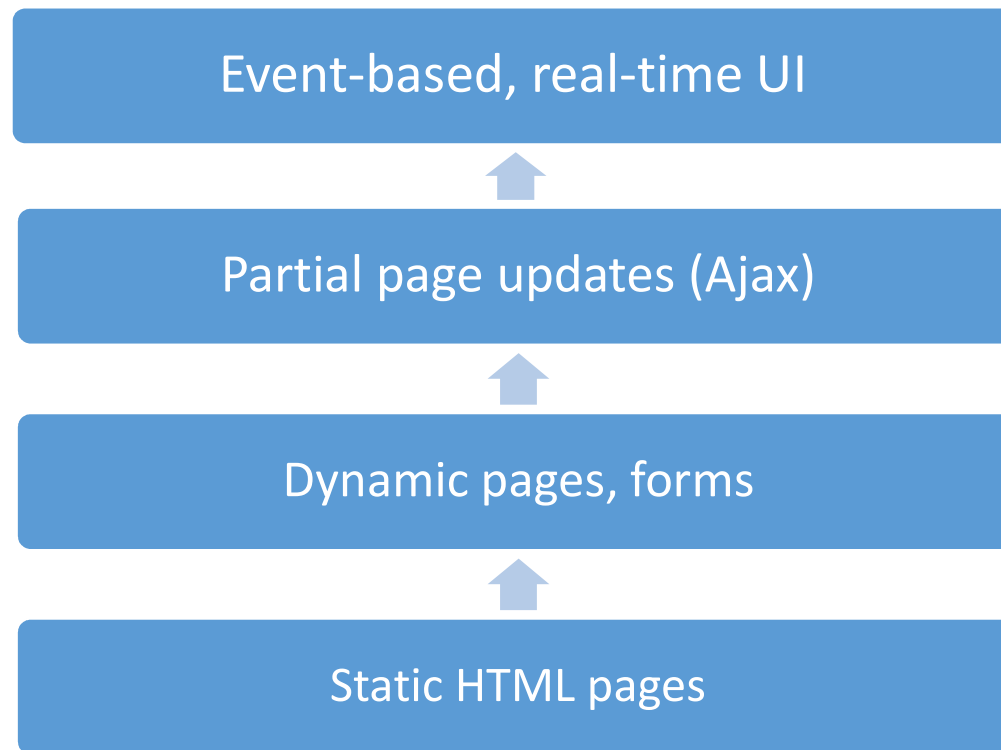


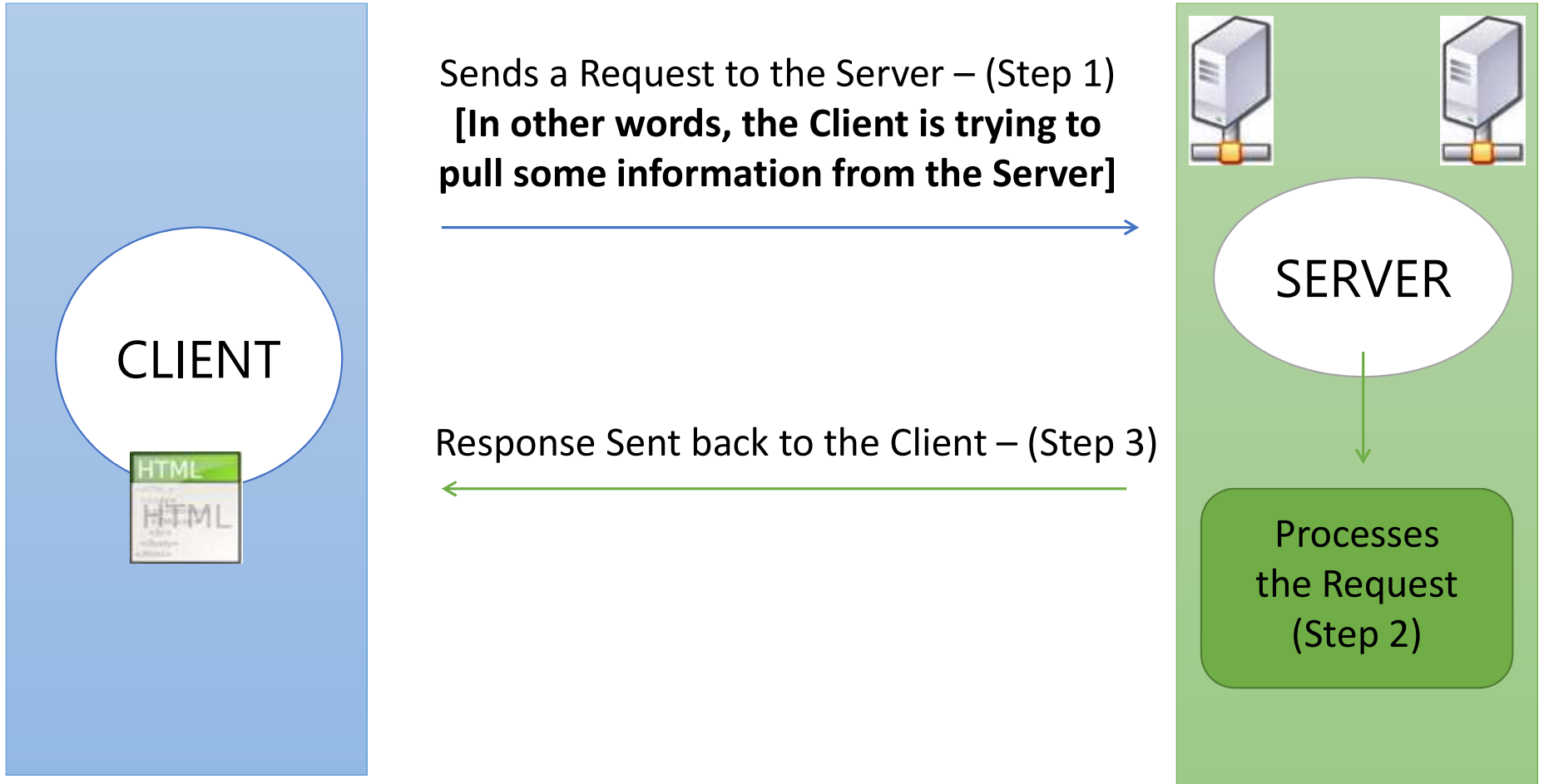
ASP.Net Core SignalR

Welcome to the Real-time world of web

Web Evolution



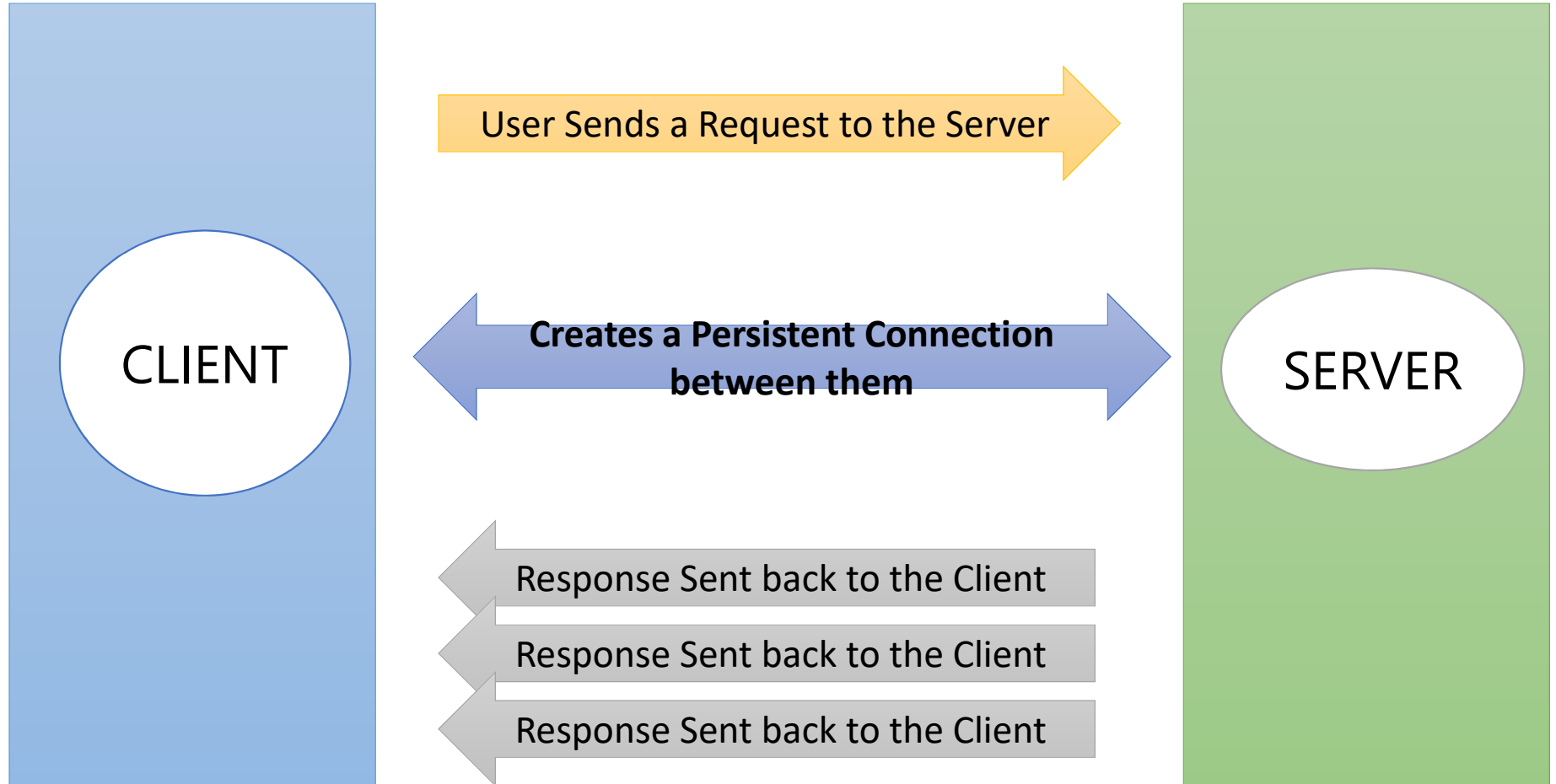
Traditional Web Approach

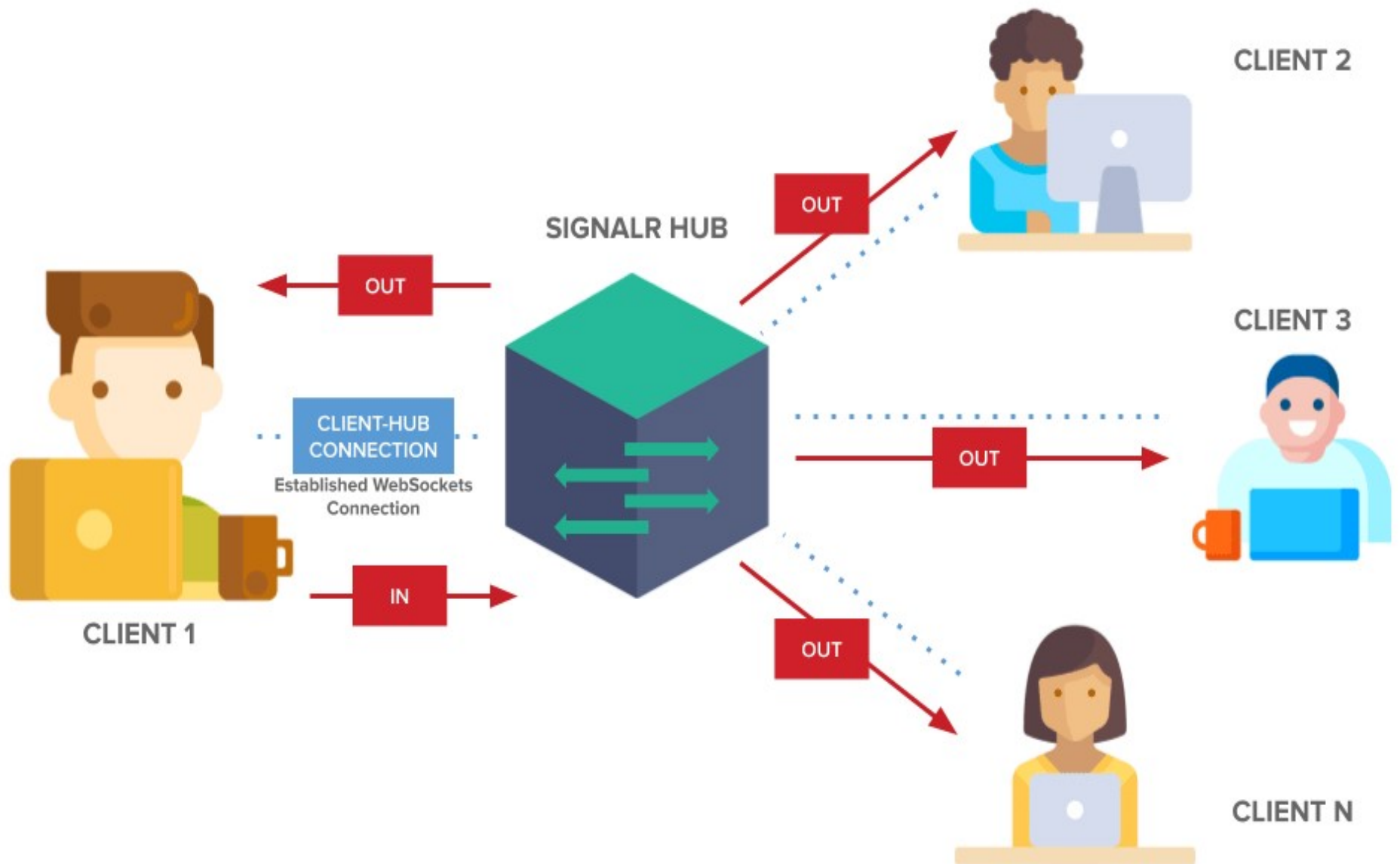


What is Real Time Web Application?

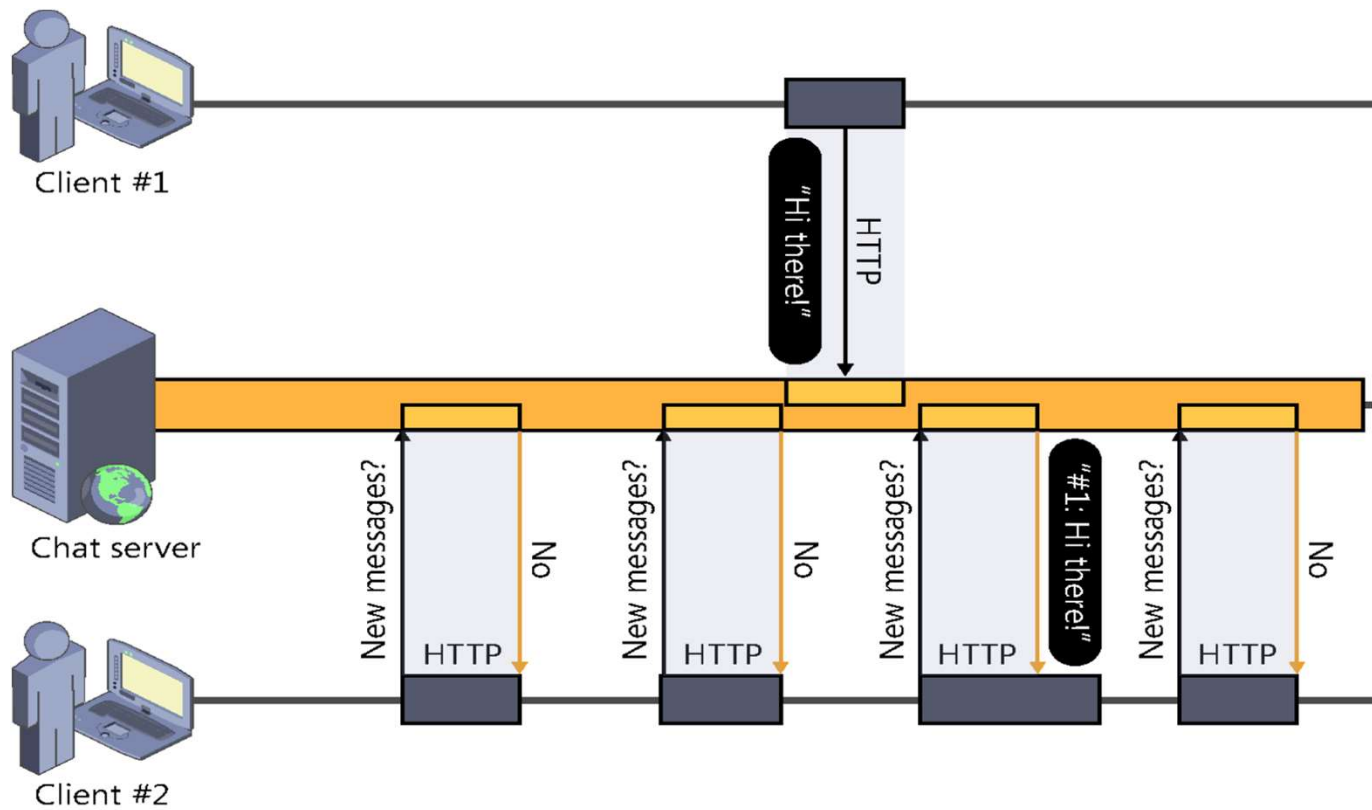
- In simple terms, “Real Time” means an immediate response being sent by the Server to the Client.
- Real Time is all about “Pushing” instead of “Pulling”
- Push Technology is completely different from Pull Technology. Its about getting told what’s new, instead of asking for what’s new!!!
- Facebook, Twitter, Yahoo Cricket Live, Stock Ticker

Real Time Web Approach

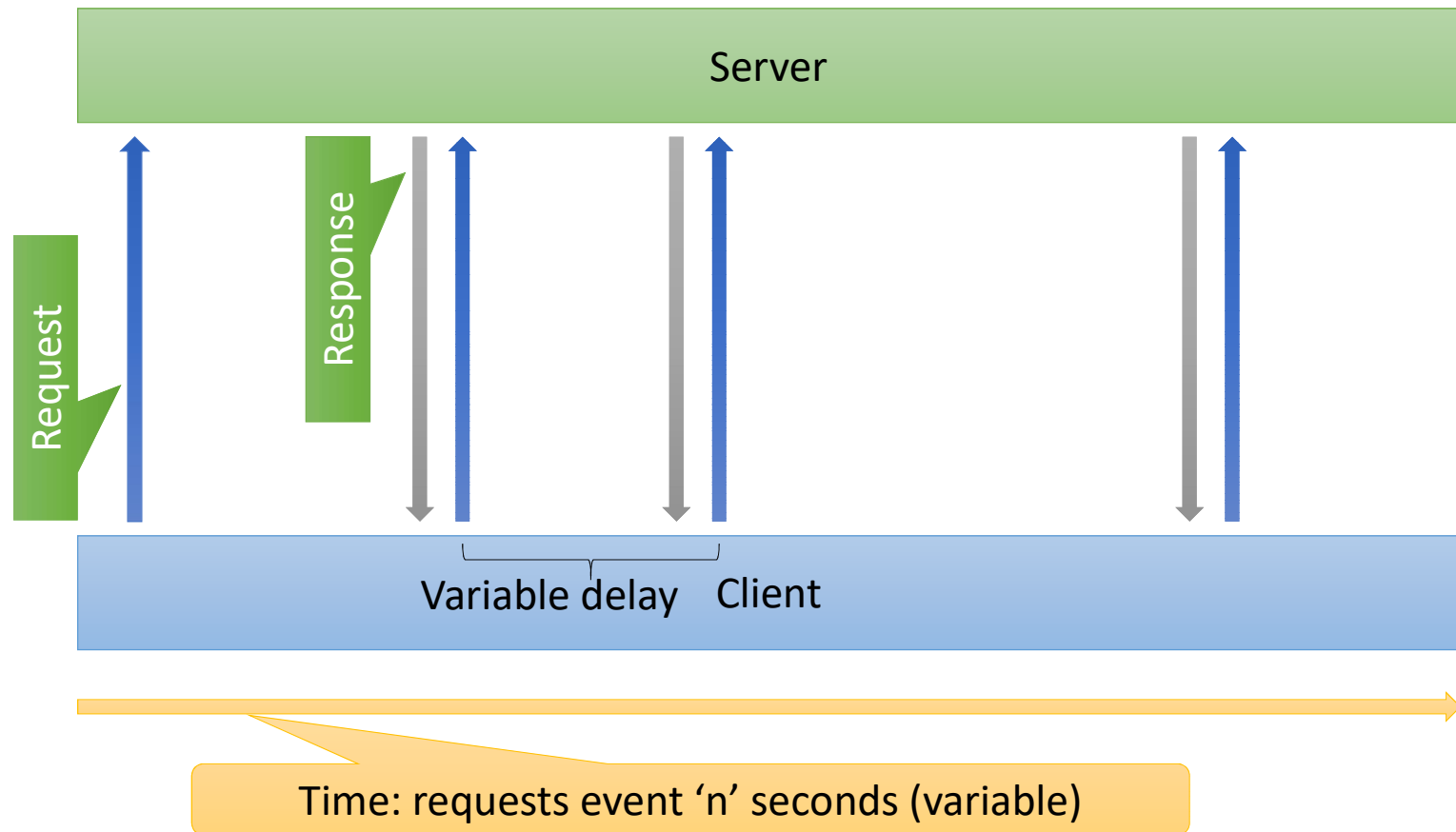




Polling



Long Polling

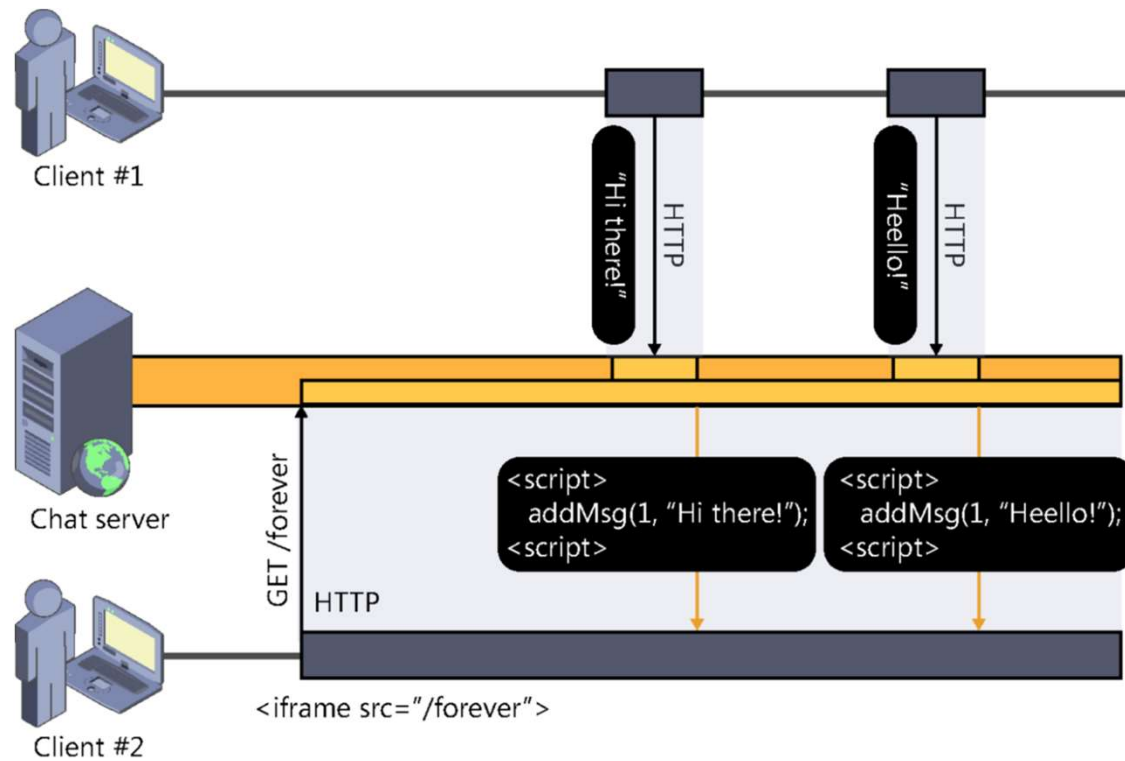


Forever Frames

- Data is sent out in chunks.
- Chunked Encoding : is the feature in the [HTTP 1.1 specification](#) allowing a server to start sending a response before knowing its total length
- A hidden iframe element is opened in the browser after page load, establishing a long-lived connection inside the hidden iframe..

Pros	Cons
Supported on IE Browser.	▪ Iframes are loaded again and again with chunks of data. ▪ All script tags remain on the page. ▪ one-way realtime connection from server to client

Forever Frames

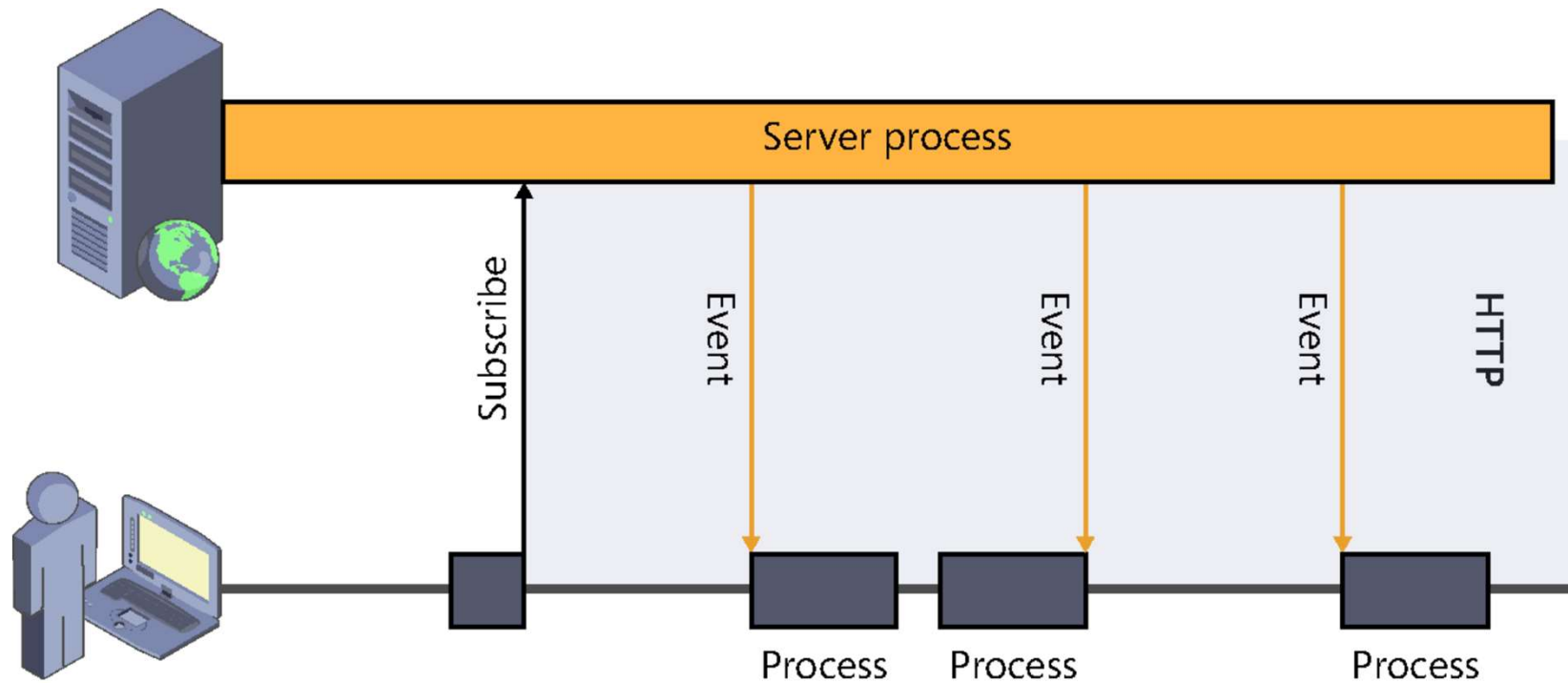


Server Sent Events

- Requires a single connection between Client-Server.
- Uses Javascript API – “EventSource” through which Client can request a particular URL to receive data stream.
- Used to send Message Notifications or Continuous Data Streams.

Pros	Cons
No need to reconnect	Works in server-to-client direction only
	Not supported in IE

Server Sent Events(SSE)



Server Sent Events(SSE)

```
<script>  
    var source = new EventSource('/elshafei/getevents');  
  
    source.onmessage = function (event) {  
        alert(event.data);  
    };  
</script>
```

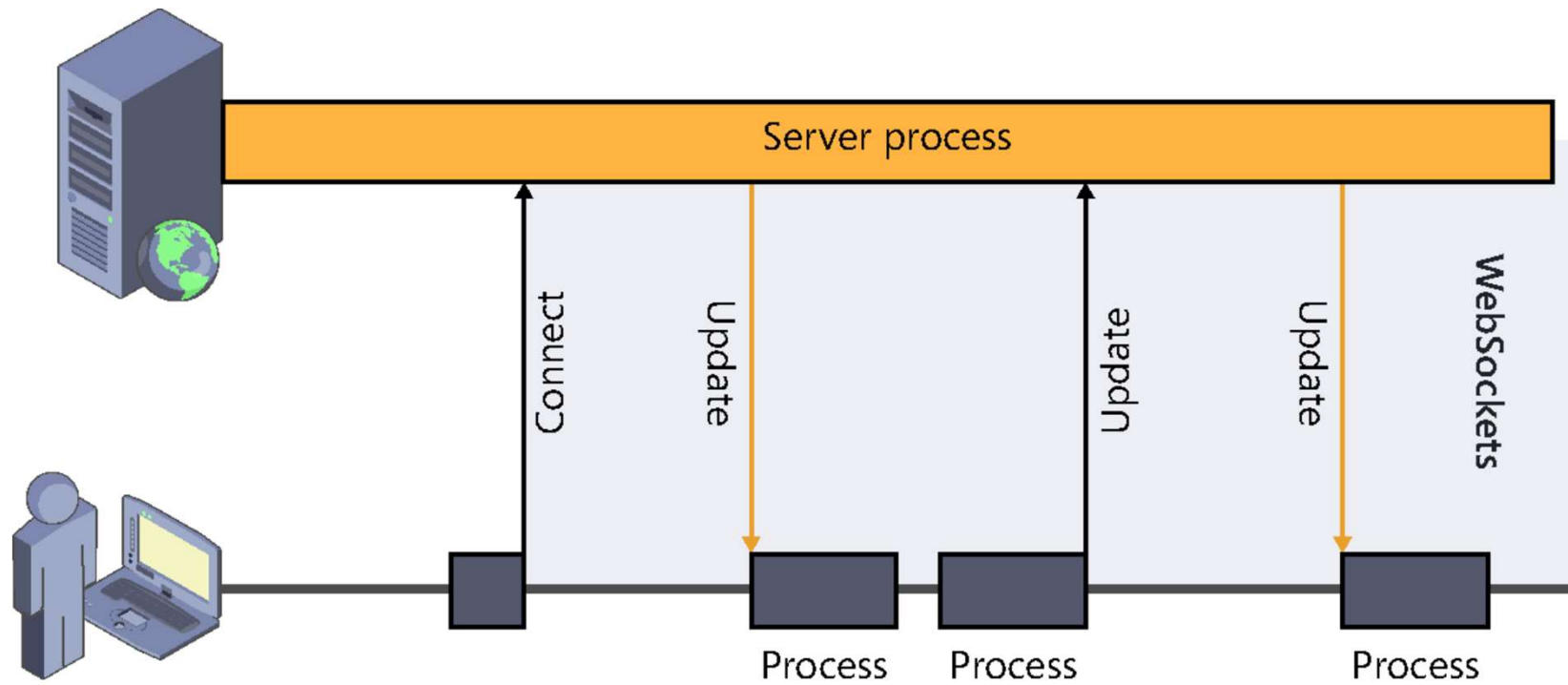
Events	Description
onopen	When a connection to the server is opened
onmessage	When a message is received
onerror	When an error occurs

WebSocket

- A new transport technique which came up with HTML5.
- a full-duplex single socket connection over which messages can be sent between client and server
- It internally works on top of TCP protocol.

Pros	Cons
Full-duplex persistent connection (both ways)	Supported only on latest browsers – (IE 10) Windows 8, Windows Server 2012 or later
Fastest solution	Works only with IIS-8.0 .Net Framework 4.5+

WebSocket



WebSocket

```
<script>
  var ws = new WebSocket("ws://localhost:9998/index");
  ws.onopen = function () {
    // Web Socket is connected, send data using send()
    ws.send("Message to send");
    alert("Message is sent...");
  };
  ws.onmessage = function (evt) {
    var received_msg = evt.data;
    alert("Message is received...");
  };
  ws.onclose = function () {
    // WebSocket is closed.
    alert("Connection is closed...");
  };
</script>
```

SignalR – Modern Web

- SignalR is an open-source library that simplifies adding real-time web functionality to apps.
- Concept initiated by “David Fowler” and “Damien Edwards”
- a server-side framework to write push services
- a set of client libraries to make push service communication easy to use on any platform
- optimized for asynchronous processing
- Open Source available on Github!!!

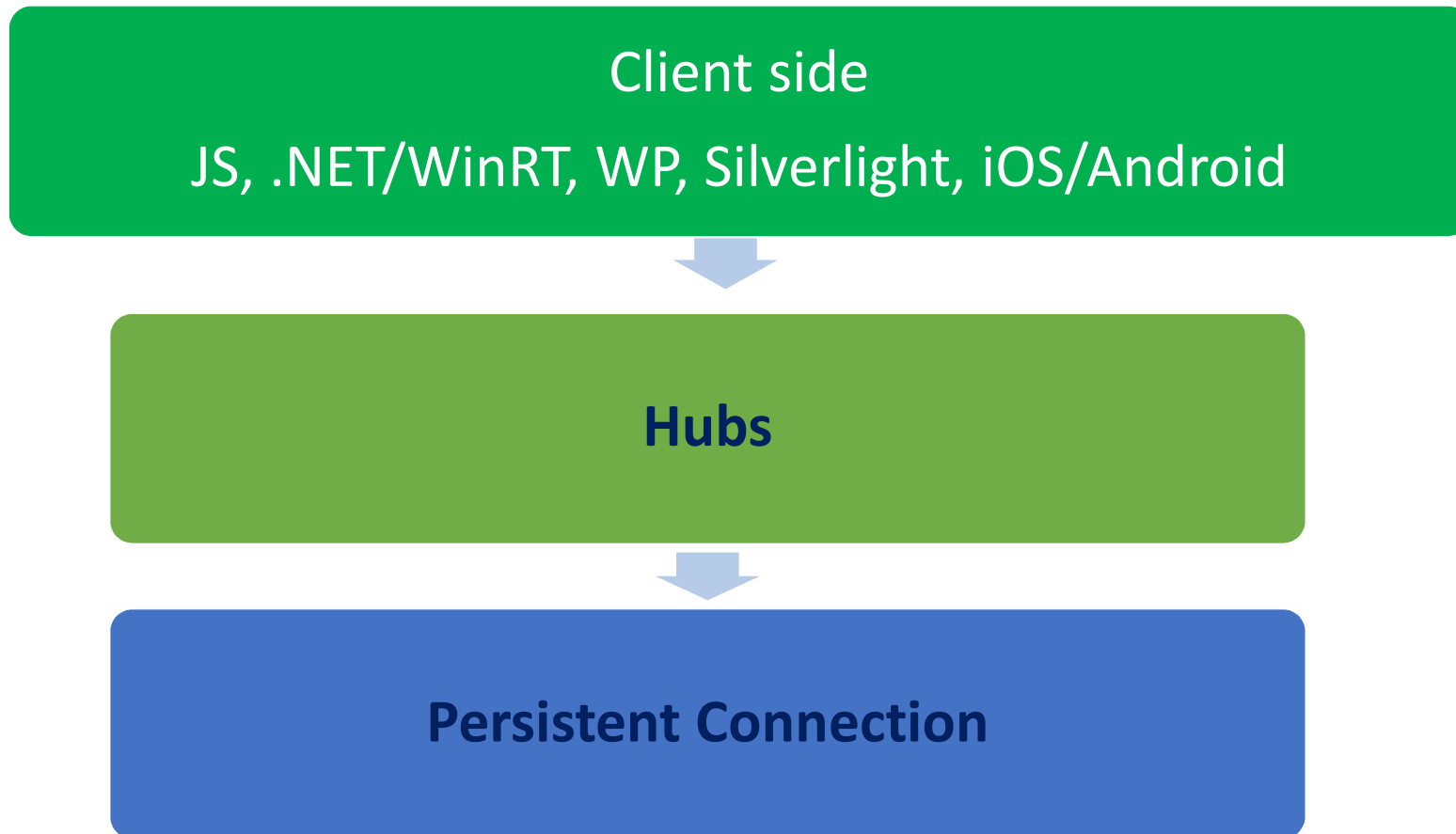
Good candidates for SignalR

- Apps that require high frequency updates from the server. Examples are gaming, social networks, voting, auction, maps, and GPS apps.
- Dashboards and monitoring apps. Examples include company dashboards, instant sales updates, or travel alerts.
- Collaborative apps. Whiteboard apps and team meeting software are examples of collaborative apps.
- Apps that require notifications. Social networks, email, chat, games, travel alerts, and many other apps use notifications.

SignalR

- **SignalR** handles connection management automatically, and lets you broadcast messages to all connected clients simultaneously, like a chat room. You can also send messages to specific clients. The connection between the client and server is persistent, unlike a classic HTTP connection, which is re-established for each communication.
- **SignalR** supports "server push" functionality, in which server code can call out to client code in the browser using Remote Procedure Calls (RPC), rather than the request-response model common on the web today.

SignalR Connections



ASP.NET Core SignalR

- **ASP.NET Core SignalR** is an open-source library that simplifies adding real-time web functionality to apps. Real-time web functionality enables server-side code to push content to clients instantly.
- SignalR provides an API for creating server-to-client ***remote procedure calls (RPC)***. The RPCs call JavaScript functions on clients from server-side .NET Core code.

Transports

SignalR supports the following techniques for handling real-time communication (in order of graceful fallback):

- WebSockets
- Server-Sent Events
- Long Polling

SignalR automatically chooses the best transport method that is within the capabilities of the server and client.

Configure SignalR(startup class)

- **ConfigureServices :**

```
services.AddSignalR();
```

- **Configure:**

```
app.MapHub<ChatHub>("/chatHub");
```

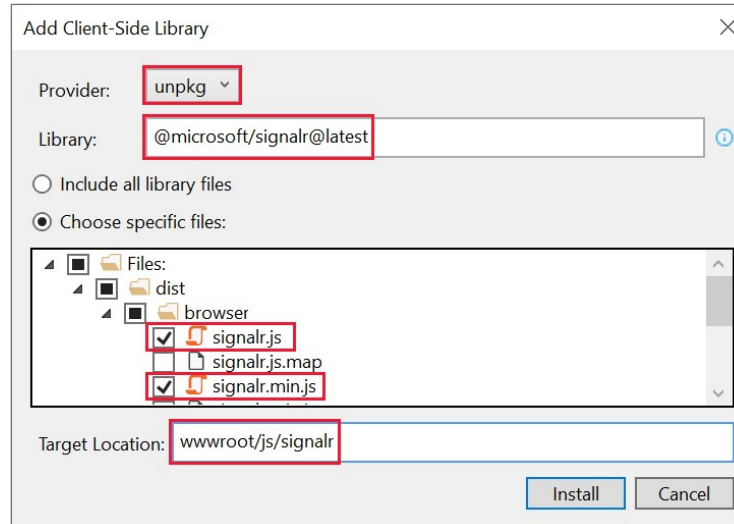
Create a SignalR hub

- In the *SignalRChat* project folder, create a Hubs folder.
- In the Hubs folder, create a *ChatHub.cs*

```
using Microsoft.AspNetCore.SignalR;  
using System.Threading.Tasks;  
  
namespace SignalRChat.Hubs  
{  
    public class ChatHub : Hub  
    {  
        public async Task SendMessage(string user, string message)  
        {  
            await Clients.All.SendAsync("ReceiveMessage", user, message);  
        }  
    }  
}
```

Add the SignalR client library

- In **Solution Explorer**, right-click the project, and select **Add > Client-Side Library**.



- **Use a Content Delivery Network (CDN)**

```
<script src="https://cdnjs.cloudflare.com/ajax/libs/microsoft-signalr/6.0.1/signalr.js"></script>
```

Add SignalR client code

```
<script src="~/js/signalr/dist/browser/signalr.js"></script>
<script>
    //define connection
    var connection = new signalR.HubConnectionBuilder()
        .withUrl("/chatHub").build();

    //start connection
    connection.start()

    //define callback fun
    connection.on("ReceiveMessage", function (user, message) {
        //anycode
    });

    //call server method
    connection.invoke("SendMessage", user, message)
</script>
```

Invoke vs send

invoke() → *Request / Response*

- Calls a server method and waits for a result
- Returns a Promise
- Supports async/await and error handling
- Use for business logic or operations requiring confirmation

```
const result = await connection.invoke("Add", 5, 3);  
console.log(result); // 8
```

send() → *Fire & Forget*

- Calls a server method without waiting for a response
- Does not return a result
- No built-in error handling
- Use for events / notifications

```
connection.send("SendMessage", "Mohamed", "Hello");
```

Configure Transport Type

If you don't specify a transport, SignalR will choose automatically:

Order: **WebSockets** → **SSE** → **Long Polling**

Force a Specific Transport

- You can explicitly choose which transport to use when creating the connection:

```
var con = new  
signalR.HubConnectionBuilder().withUrl("/chat",signalR.HttpTransportType.ServerSentEvents).build();
```

Available Transport Options

- `signalR.HttpTransportType.WebSockets`
- `signalR.HttpTransportType.ServerSentEvents`
- `signalR.HttpTransportType.LongPolling`

Protocol-Based Messaging

- SignalR does **not call methods directly**
It uses **protocol-based messages** over a persistent connection
- Communication = **Serialized Messages (RPC style)**

How It Works

1-Client calls:

```
connection.invoke("SendMessage", "Mohamed", "Hello");
```

2- Converted to message:

```
{  
  "type": 1,  
  "target": "SendMessage",  
  "arguments": ["Mohamed", "Hello"],  
  "invocationId": "1"  
}
```

3-Sent via transport (WebSocket / SSE / Long Polling)

4- Server:

Matches target → Hub method

Binds arguments → executes method

JavaScript Client Logging

Configure logging when creating the connection:

What You Can See in Logs

Connection start / stop ,Transport selection (WebSockets / fallback), Messages sent via invoke / send , Server responses ,Reconnect events and failures

Logging Levels

- Trace → Full protocol details
- Debug → Development-level details
- Information → Connection lifecycle + key events
- Warning → Non-fatal issues
- Error → Failures only
- None → Disable logging

```
const connection = new signalR.HubConnectionBuilder()  
    .withUrl("/chatHub")  
    .configureLogging(signalR.LogLevel.Information)  
    .build();
```